Gas pipeline construction

A respected company is building an underground gas pipeline to the People's Republic of China. At the first stage of work, a trench is dug and pipes are laid into it (welding of seams is not performed at this stage). The builders have at their disposal a package of pipes of various lengths (1 to 40 meters, the length of each is an integer number) and a multifunctional CNC pipelayer capable of digging 20 meters of a trench (or lengthening the existing one by this amount) at the command of the control program. A package of pipes is loaded into the pipelayer, the machine gets one pipe out of the package, measures its length and lays it in the trench, starting from a distance from the beginning of the trench as determined by the control program. In this case, each pipe is assigned PID, a unique number in the order of getting the pipe from the package (starting from 1). Also, the pipelayer can remove a pipe from the trench by its PID. Due to the extremely high risk of damage, it is prohibited to move the laid pipes inside the trench.

As the project is very important for the company, the Quality Control Team comes periodically and inspects the laid pipes (during this time the pipe-laying is suspended). The team may indicate PIDs of one or more pipes damaged during laying, which must be removed and sent for recycling; then the pipelaying resumes. As soon as the pipe package is empty, the Performance Evaluation Team arrives immediately. It calculates KPI equal to the ratio of the total sum of the lengths of the pipes laid in the trench to the length of the dug trench. Your task is to write a control program for the pipelayer.

Interaction Protocol

This is an interactive task, and instead of file input-output you will be working with a special program, called interactor. You will interact with this program using the standard input-output streams.

An interactor gives you instructions from script one by one. Instructions can be the following integer numbers:

L – means to lay the pipe of the given length $L \in [1,40]$,
$-K$ – means to remove the pipe which PID is K from the trench,
0 – special command that marks the end of the sequence.

In response to each command your solution should provide one or several instructions specifying what to do next.

Possible output instructions:

1. If the input instruction was an instruction for laying the next pipe, your answer should be a couple of integer numbers: **N D**, where:
   - $N \geq 0$ – means that to dig (lengthen) a trench by N*20 meters,
   - $D \geq 0$ – means to lay the pipe at the given distance D from the beginning of the trench.
2. If the input instruction was an instruction for removing a pipe, your solution should response with distance from the beginning of the trench to the pipe being removed,
3. If the input instruction is "0" no response is needed and the program should be stopped immediately.

It is guaranteed that total number of input commands does not exceed 200 000 in any test scenario.

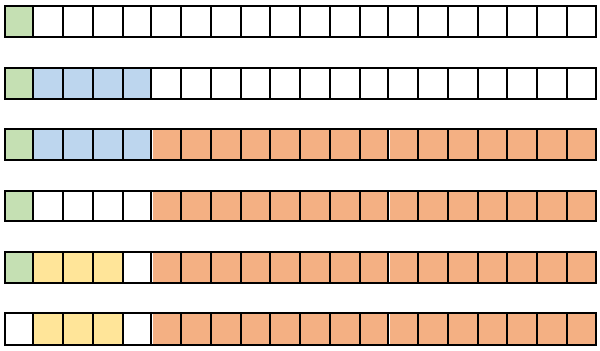
Make sure you print the end-of-line character and flush the output stream buffer (function flush of the programming language) after each printed output instruction. Otherwise the solution can get Timeout response.

Example

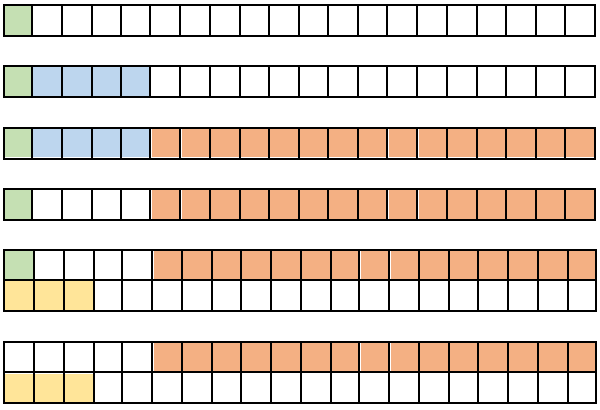
For reading convenience, commands in the example are separated by empty lines. Your program must not print empty lines.

Here are two different examples of the interaction process with the same input instructions sequence.

First one:

standard input stream	standard output stream	Illustration
1 4 15 -2 3 -1 0	1 0 0 1 0 5 1 0 1 0	

And alternative possible output sequence:

standard input stream	standard output stream	Illustration
1 4 15 -2 3 -1 0	1 0 0 1 0 5 1 1 20 0	

Example explanation

The input sequence in both samples means to get out of the package a 1-meter pipe and lay it; do the same for pipes 4 and 15 meters long, sequentially; remove the pipe with PID=2 from the trench (that is, the second pipe got from the package), get and lay a 3-meter pipe, remove the pipe with PID=1.

--

The solution is **invalid** if any instruction of laying the pipe is impossible to perform due to not enough room starting from the specified distance from the beginning. Both output sequences above are **valid**.

However, these solutions have different **KPIs**. KPI of the first sequence is 0.9, and the second is 0.45. So, for this input sequence the first strategy is better.

In case of getting invalid command, interactor will stop immediately and return an error message describing what exactly is wrong. KPI for **invalid** solutions is always 0.

There is also a limit for the length of trench you can dig. The maximum length of trench is 100 000 000 meters, in case of excessing this length the solution becomes invalid.

Judgment

Solutions are tested on 30 different test scenarios.

15 of them are **open tests** provided to participants, so they can check their solution locally. 15 other tests are **closed tests** and solutions will be tested on them at the end of contest.

Each test has a weight that Is used to calculate the final score:

Test #	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Weight	0.1	1										4			10
Openness	Open														

Test #	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
Weight	4											10			
Openness	Closed														

Also, during the contest participants can send their solution to NSUts and get the results for all open tests. Participants can send their solutions multiple times without any penalties. However, only the last attempt will be used to calculate the final score.

The final score of the solution is a sum of KPIs (as defined above) that solution shows on every test (both open and closed tests are used), multiplied by the corresponding weight from the table above:

$$Score = \sum_{i=1}^{30} KPI_i * weight_i$$

A solution wins if it gets the maximal score. If several solutions yield the same score, the winner is selected by a minimal total execution CPU time on these tests.

Note that in most tests the maximum possible KPI is close to 1.

Limits

Solution must handle the whole sequence in less than 5 seconds of CPU time and consume less than 256 Mb of RAM.

NSUts

You can find a list of the all your submissions on the «Результаты» page in the nsuts judging system.
On that page two links «Открыть отчёт о тестировании» and «Скачать результаты тестирования» will be available for the each tested submission.

Following the first link you can find a detailed report about the judgment: status, KPI, consumed cpu time and score for the each test.

Following the second link you can download a zip-archive, which contains your outputs for the each test and an interactor error message in case your program violates the interaction protocol or gives invalid command.

Materials

You can download an archive with all needed materials (open tests and interactor) can be downloaded from NSUts. It consists of the following parts:

File name	Description
tests/*.in	These are open tests: sequences of input instructions that will be given to your solution. Feel free to analyze them in seek of hints about of what sort of input data can be issued.
interactor.cpp testlib.h	These are source code of the interactor: special program used to test your solution on the given set of open tests.
interactor.exe	Pre-built interactor executable for testing your solution locally on Windows systems.
interactor	Pre-built interactor executable for testing your solution locally on Linux systems.
runner.py	Python script that should be used to run local tests on your solution.

To run local tests:

1. Make sure you have Python 3.5 or later installed on your local machine
2. Run 'python runner.py -d tests <your_solution_binary>' to run your solution over all tests in "tests" directory. Wrap few words command in quotes 'python runner.py -d tests "java -jar solution.jar"' to run java solution
3. Run 'python runner.py -i tests/1.in <your_solution_binary>' to run your solution on "tests/1.in" instead